

Documentação Técnica: Sistema de Visão ZoyBlocks + Cozmo

Este documento descreve a arquitetura de visão, desde o reconhecimento de imagem no Python até a execução de blocos de lógica no navegador.

Pré-requisitos e Instalações (O Motor)

Para que o Cozmo "enxergue", utilizamos a biblioteca **OpenCV** integrada ao SDK do Cozmo. No ambiente Python do servidor, é necessário garantir as seguintes bibliotecas:

- **opencv-python**: Para processamento de imagem e detecção de faces.
- **pyzbar**: Para a leitura e decodificação de QR Codes.

Comando de instalação:

Bash

```
pip install opencv-python pyzbar
```

Passo 1: Ensinar o Python a reconhecer (O Olho)

No script Python (`cozmo_server.py`), criamos uma rotina que processa cada frame da câmera do robô. O Python não apenas exibe a imagem, mas analisa dados:

1. **Detecção de Face**: Utilizamos o classificador *Haar Cascade* do OpenCV para identificar coordenadas de rostos.
2. **Detecção de QR Code**: A biblioteca `pyzbar` escaneia a imagem em busca de padrões de código de barras/QR.
3. **Envio de Eventos**: Em vez de enviar apenas a imagem, o Python envia um objeto JSON chamado `vision_event`:

JSON

```
{
  "type": "vision_event",
  "face": true,
  "qrcode": "Texto_do_QR"
}
```

Passo 2: A Memória Global (O Cérebro no Electron)

Um dos pontos cruciais para o funcionamento foi a criação de uma **Variável Global** no `home.js`. Como o Blockly e o Listener de Logs rodam em momentos diferentes, eles precisam de um "quadro de avisos" compartilhado.

No topo do `home.js`, definimos:

```
JavaScript
window.estadoVisao = {
  rostoDetectado: false,
  ultimoQR: ""
};
```

Por que Global (`window`)? Isso permite que o Console do Desenvolvedor, o sistema de logs e o motor do Blockly acessem o mesmo dado em tempo real sem conflitos de escopo.

Passo 3: Criação do Bloco de Visão (A Interface)

Criamos o arquivo `cozmo_vision.js` para definir como o usuário interage com essa inteligência.

- **O Bloco:** Criado com um `Dropdown` para escolher entre "Rosto" ou "QR Code".
- **O Gerador:** O bloco não executa ação de movimento, ele **retorna um valor booleano** (Verdadeiro ou Falso).

Código do Gerador:

```
JavaScript
cozmoGenerator.forBlock['cozmo_check_vision'] = function(block) {
  const tipo = block.getFieldValue('TIPO');
  const code = `await checarVisao("${tipo}")`; // Chama a ponte criada no home.js
  return [code, cozmoGenerator.ORDER_ATOMIC];
};
```

Passo 4: A Ponte de Execução (`checarVisao`)

Para que o código gerado pelo Blockly `await checarVisao("FACE")` funcione, implementamos a função de ponte no `home.js` dentro da inicialização do app.

Função Implementada:

JavaScript

```
window.checarVisao = async function(tipo) {  
  if (!window.estadoVisao) return false;  
  
  if (tipo === "FACE") {  
    return window.estadoVisao.rostoDetectado;  
  } else if (tipo === "QR") {  
    return window.estadoVisao.ultimoQR !== "";  
  }  
  return false;  
};
```

Esta função é o "tradutor" que o Blockly usa para perguntar à memória global se o robô está vendo algo naquele exato momento.

Fluxo de Funcionamento (Resumo)

1. **Câmera:** O Cozmo envia o frame para o Python.
 2. **Processamento:** OpenCV detecta um rosto e envia `vision_event` via IPC.
 3. **Captura:** O `home.js` recebe o evento e atualiza `window.estadoVisao.rostoDetectado = true`.
 4. **Lógica:** O bloco de "Se" no Blockly chama `checarVisao("FACE")`.
 5. **Ação:** A função retorna `true` e o Cozmo executa o comando dentro do "Se" (ex: piscar luzes).
-

Dicas de Debug no Console

Para testar o sistema sem precisar do robô físico, você pode digitar no console (F12):

- `window.estadoVisao.rostoDetectado = true;` (Simula que um rosto foi visto).
 - `console.log(window.estadoVisao);` (Verifica o que o robô está sentindo agora).
-

Status da Implementação: ☒ Operacional **Próximo Passo Sugerido:** Criar blocos de "Esperar até ver rosto" para comportamentos autônomos.

Deseja que eu detalhe mais algum desses passos ou salve este documento em um formato específico?